

Projet INFO-F2O3: Recherche du plus court chemin dans un réseau de transport

Matteo Morbée - 549684
Université Libre de Bruxelles

18 mai 2025

Résumé

Ce rapport présente une solution complète au problème de recherche de plus courts chemins dans un réseau de transport multimodal en Belgique. Le système repose sur une structure de données adaptée, une modélisation rigoureuse, et des algorithmes efficaces prenant en compte différentes variantes du problème. Ce travail s'inscrit dans le cadre d'un projet d'informatique du cours d'algorithmique de deuxième année.

Table des matières

1	Introduction	3
2	Structure de données utilisée	3
2.1	Description des structures de données	3
2.1.1	Trip et Route	3
2.1.2	Stop et Neighbour	4
2.1.3	<i>parsing</i> des stop_times	4
2.1.4	PathEdge	5
3	Construction du graphe	6
4	Algorithmes de plus courts chemins	6
4.1	Algorithme principal	6
4.1.1	Structure de l'algorithme	6
4.1.2	Étude de l'algorithme	7
4.2	Correctitude et optimalité	8
4.2.1	Comportement de l'algorithme sans heuristique (Dijkstra)	8
4.2.2	Comportement de l'algorithme avec heuristique (A*)	8
4.3	Analyse de complexité	11
4.3.1	Complexité temporelle de l'algorithme	11
4.3.2	Complexité spatiale de l'algorithme	12
4.3.3	Complexité temporelle et spatiale du programme	12
5	Gestion des variantes du problème	12

6	Modélisation de la marche	13
7	Tests	13
8	Conclusion	15

1 Introduction

Dans ce projet, nous nous intéressons à la recherche de plus courts chemins dans le réseau de transport public en Belgique. Nous utilisons une version revisitée de l'algorithme A* pour répondre efficacement aux requêtes de plus courts chemins entre deux arrêts de transport public à une heure donnée. Cet algorithme a été développé autour d'un benchmark réfléchi, de comparaisons et de recherches dans plusieurs directions possibles. Le choix d'un algorithme de type A* fût un mélange de challenge et de fascination pour son exécution. L'équilibre entre l'algorithme de **Dijkstra** et l'algorithme **Greedy Best-First Search** nous a particulièrement intrigué. Nous verrons tout au long de ce rapport les avantages mais aussi les contraintes d'un tel choix d'implémentation.

Nous verrons aussi en plus de notre algorithme, l'implémentation de nos structures de données, essentielles au bon déroulement de l'algorithme, surtout avec un nombre de données relativement élevé. Mais aussi comment nous parvenons à récupérer efficacement et trier toutes les données brutes des fichiers CSV du dossier GTFS comprenant des données réelles (bien que simplifiées dans notre cas) sur les transports publics belges.

2 Structure de données utilisée

Nous avons plusieurs objets créés lors du traitement des fichiers CSV. Certains font simplement office de *wrapper class* tandis que d'autres sont des classes primordiales au bon déroulement de l'algorithme. Nous allons passer en revue toutes ces structures de données (disponible dans le package `Java Models` du code source).

2.1 Description des structures de données

2.1.1 Trip et Route

Ces deux structures de données sont relativement liées. La première, **Trip** sont des trajets différenciés par un **Id** unique et sont chacun reliés à un **routeId**, nous permettant d'associer une route à chacun d'entre eux. Ces **Trip** nous permettrons entre autre de détecter des segments continus et des transferts. Ils nous seront aussi utiles lors du *parsing* des **stop_times** que nous verrons en détail au point 2.1.3.

TABLE 1 – Classe Trip

Attribut	Description
id	Identifiant du trajet
route	Objet Route associé

TABLE 2 – Classe Route

Attribut	Description
id	Identifiant de la ligne
name	Nom court (e.g. 71)
longName	Nom complet de la ligne
transportType	Type (BUS, METRO, etc.)

2.1.2 Stop et Neighbour

Ces deux classes sont le centre même de notre structure de données. **Stop** (un arrêt) représente virtuellement un noeud de notre futur graphe. C'est grâce à ces objets que nous pourrions itérer dans notre algorithme, principalement grâce à **neighbours**, une liste de tous les voisins accessibles en transport ou à pied (Voir point 6 pour une explication détaillée de la façon dont nous déterminons les voisins accessibles à pied)

Nous observons que nous n'enregistrons pas directement les objets **Stop** dans nos listes. En effet, nous passons par une *wrapper class* nommée **Neighbour**. Cette classe nous permet d'obtenir toutes les informations de la liaison entre l'arrêt et son voisin. Nous verrons lors des explications sur l'implémentation de l'algorithme pourquoi nous avons fait le choix de séparer les trajets à pieds des trajets en transport.

TABLE 3 – Classe **Stop**

Attribut	Description
id	Identifiant unique de l'arrêt
name	Nom de l'arrêt
lat	Latitude de l'arrêt
lon	Longitude de l'arrêt
neighbours	Voisins accessibles par transport ou à pied
index	indice unique de l'arrêt

TABLE 4 – Classe **Neighbour**

Attribut	Description
stop	Arrêt voisin
trip	Trajet associé
fromTime	Heure de départ depuis l'arrêt courant (-1 si trajet à pied)
time	Durée pour atteindre le voisin

2.1.3 *parsing* des stop_times

Nous nous attardons sur ce *parsing* pour deux raisons. Premièrement, les fichiers **stop_times.csv** sont de loin les plus volumineux, nous demandant une gestion de la mémoire accrue. Deuxièmement, c'est ici que nous allons associer les arrêts entre eux grâce aux données contenues dans les fichiers CSV.

Pour ce premier argument, nous avons d'abord ajouté une *wrapper class* nommée assez simplement **StopTime**. à chaque ligne lue du CSV, nous créons un objet **StopTime** que nous ajoutons dans une liste, elle même contenue dans une **HashMap** dont la clé est l'objet **Trip** correspondant. Cela nous permet de limiter le nombre d'objet créé et d'objets en mémoire. En effet, excepté pour les attributs **departure** et **sequence**, nous ne faisons que récupérer les objets **Trip** et **Stop** déjà existants (voir table 5).

Ensuite, lorsque tous les fichiers ont été parsés et que les **StopTime** ont été placés correctement dans notre **HashMap**, nous ratissons ceux-ci pour associer à chaque arrêt leurs voisins respectifs.

En termes de complexité temporelle, le *parsing* des fichiers est proportionnel au nombre total de lignes L qu'ils contiennent. Les opérations réalisées pendant cette phase, telles que l'ajout dans une **HashMap** ou dans une **ArrayList**, sont effectuées en temps constant,

soit $\mathcal{O}(1)$. Ainsi, la complexité globale du *parsing* est $\mathcal{O}(L)$.

Lors de la phase de restructuration et d'association des arrêts, nous devons traiter T objets de type **Trip**. Pour chaque trip, nous effectuons les opérations suivantes :

- Un tri de la liste des S objets **StopTime**, ce qui coûte $\mathcal{O}(S \log S)$;
- Une itération linéaire sur cette liste pour créer des objets **Neighbour**, ce qui coûte $\mathcal{O}(S - 1) = \mathcal{O}(S)$.

La complexité temporelle totale pour cette phase est donc $\mathcal{O}(T \cdot (S \log S + S)) = \mathcal{O}(T \cdot S \log S)$.

La complexité spatiale est dominée par la création des objets **Neighbour**. Pour chaque trip contenant S arrêts, on crée exactement $S - 1$ voisins. Ainsi, la complexité spatiale est $\mathcal{O}(T \cdot (S - 1)) = \mathcal{O}(T \cdot S - T)$.

Bien que cette différence soit négligeable d'un point de vue asymptotique (puisque $\mathcal{O}(T \cdot S - T) \in \mathcal{O}(T \cdot S)$), elle existe bel et bien, et il est pertinent de la mentionner.

TABLE 5 – Classe **StopTime**

Attribut	Description
stop	Arrêt concerné par l'horaire
departure	Heure de départ à cet arrêt
sequence	Ordre de passage dans le trajet

2.1.4 PathEdge

Cette dernière structure de donnée est uniquement utilisée dans l'algorithme et dans la reconstruction du chemin et son affichage. C'est la représentation virtuelle de notre graphe construit par l'algorithme. Cette classe détient l'arrêt de départ et d'arrivée ainsi que le temps auquel on quitte ce premier arrêt et le temps auquel on arrive au deuxième arrêt. Pour pouvoir *print* efficacement le chemin grâce à une création de segments, nous gardons aussi un **Trip** associé à l'arrêt de départ.

TABLE 6 – Classe **PathEdge**

Attribut	Description
fromStop	Arrêt de départ de l'arête
toStop	Arrêt d'arrivée de l'arête
trip	Trajet utilisé pour ce déplacement
departureTime	Heure de départ depuis l'arrêt de départ
arrival	Heure d'arrivée à l'arrêt d'arrivée

3 Construction du graphe

La construction du graphe dans lequel notre algorithme va itérer se fait naturellement lors du *parsing*. En effet, après avoir initialisé les objets structurels (**Stop**, **Trip**, **Route**), nous ne faisons que relier chaque arrêts entre eux grâce à **Neighbour** et au *parsing* de **stop_times**.

Bien que cela soit suffisant, il nous manque la structure de donnée **PathEdge** pour pouvoir retracer correctement le chemin.

Pour déterminer l'arrêt de départ et d'arrivée, nous avons décidé de créer des *super* arrêt. Dans la méthode **findStopByName**, nous récupérons tous les candidats possibles pour les merge en un seul **Stop**, le choix de la position de ce *super* arrêt est arbitraire, nous prenons automatiquement les données du premier candidat de la liste mais ajoutons un suffixe *-SUPER* à l'identifiant pour les différencier. Ensuite, nous faisons un *merge* de tous les voisins de tous les arrêts candidats (grâce à **HashSet**, les doublons ne sont pas insérés). Cette opération nous permet de **toujours** trouver le meilleur départ, au prix d'une complexité $\mathcal{O}(S)$ avec S le nombre d'arrêts et d'une approximation (négligeable) de la position de l'arrêt initial et d'arrivée.

Chaque arête a un coût strictement positif (obligatoire pour notre algorithme A^*) égal au temps parcouru entre les deux (pour le cas des déplacements à pied, se référer à la section 6).

4 Algorithmes de plus courts chemins

4.1 Algorithme principal

4.1.1 Structure de l'algorithme

Nous avons décidé de développer notre algorithme autour de l'algorithme A^* théorique [6]. L'algorithme est composé de plusieurs conteneurs :

- **indexToStopMap** Cette *map* permet de retrouver les arrêts par leur indice, qui nous est retourné lorsque nous récupérons la clé minimale de **IndexMinPQ** avec **delMin()**. Cette map augmente donc proportionnellement avec le nombre d'arrêts rajoutés à la queue de priorité.
- **openQueue** Aussi appelée *frontière*, cette queue de priorité est celle qui va décider le prochain arrêt à analyser selon sa priorité.
- **closedSet** Ce *set* contient tous les arrêts qui ont déjà été parcourus, nous évitant des calculs inutiles.
- **costSoFar** Cette *map* permet d'associer à chaque arrêt son coût réel pour y arriver. Au départ, la *map* est vide. Si un arrêt n'est pas dans cette liste, nous pouvons considérer qu'il n'a jamais été atteint et nous pouvons lui associer un nouveau coût.
- **edgeData** Cette dernière *map* associe à chaque arrêt son arête entrante. Cela permet de retracer facilement le chemin.

Ces conteneurs sont primordiaux. En effet, nous avons fait le choix d'avoir nos objets immutables, pour des raisons de simplicité au cas où l'algorithme doit être relancé plusieurs fois et ne pouvons donc pas enregistrer des valeurs modifiables dans celles-ci (p.e, le coût pour arriver à un noeud, **costSoFar**, aurait pu être placé directement dans **Stop**).

4.1.2 Étude de l'algorithme

La boucle principale de notre algorithme est ordinaire, comme pour tout algorithme, nous récupérons l'arrêt à plus basse priorité pour analyser tous les chemins possibles depuis celui-ci. Si cet arrêt a déjà été parcouru, nous le passons, sinon, nous vérifions d'abord si nous sommes arrivés, et seulement après nous exécutons le travail de recherche.

Algorithm 1: searchPath

```

1 Data:
2 departure, destination, departureTime;
3 Result:
4 Path from departure to destination;
5 resetSearch(); this.destination ← destination;
6 while !openQueue.isEmpty() do
7   | current ← getLowestPriorityStop();
8   | if current is already visited then
9   |   | continue;
10  | end
11  | markAsVisited(current);
12  | if current is destination then
13  |   | return reconstructPath(current);
14  | end
15  | expand(current);
16 end
17 return no path found;
```

Voici le déroulement d'une recherche sur les arrêts voisins :

Nous itérons sur les voisins accessibles en transport ou à pied dans `processNeighbours`. Avant tout calcul, nous devons vérifier que le trajet est à pied pour pouvoir associer le temps de départ à celui auquel nous arrivons sur l'arrêt. Ensuite, nous pouvons déjà ignorer les arrêts qui sont atteints avant le temps de départ de l'arrêt étudié, et nous calculons le temps d'attente. Ce temps d'attente associé au temps de déplacement entre les deux arrêts est considéré dans notre algorithme comme le coût réel entre deux arrêts. Nous calculons grâce à ce coût et au temps d'arrivée sur l'arrêt étudié le temps auquel nous arrivons à l'arrêt voisin.

Algorithm 2: processNeighbours

```

1 Data:
2 current;
3 for neighbour ∈ current.neighbours() do
4   | if neighbour is reachable by walking then
5   |   | departureTime ← current.arrivalTime();
6   | end
7   | if departureTime < current.arrivalTime() then
8   |   | continue;
9   | end
10  | calculateWaitingTimeAndTravelTime();
11  | tryToExpand(current, neighbour);
12 end
```

Ensuite, nous faisons les vérification nécessaires à l'expansion du graphe ou non dans

tryExpand. Cette fonction calcule le coût réel g du chemin depuis le départ et le compare au coût existant de l'arrêt voisin. Si ce coût est meilleur que l'actuel, nous pouvons accéder à la dernière étape de notre itération.

Algorithm 3: tryToExpand

```

1 Data:
2 current, neighbour;
3 if neighbour has better cost then
4   | calculateHeuristicAndPriority(neighbour);
5   | updateNeighbourInQueue(neighbour);
6 end
```

Cette dernière étape n'est pas des moins importantes puisque c'est elle qui détermine l'**heuristique** et donc la priorité de l'arrêt voisin.

4.2 Correctitude et optimalité

4.2.1 Comportement de l'algorithme sans heuristique (Dijkstra)

Nous pouvons nous demander comment notre algorithme se comporterait si seul le coût total du noeud était compris dans la priorité de celui-ci. Cela reviendrait à appliquer **Dijkstra** [9]. Nous avons pu observer sur un échantillon de dix voyages que notre algorithme correspond en effet à **Dijkstra**, avec une forte augmentation du nombre de noeuds visités selon la distance du voyage (table 7). Nous pourrions observer les résultats selon les variants à la section 5.

4.2.2 Comportement de l'algorithme avec heuristique (A*)

Nous allons observer le même échantillon mais cette fois-ci avec notre heuristique. Mais tout d'abord, nous allons expliquer son choix.

L'heuristique est déterminée par l'estimation du temps restant depuis l'arrêt voisin $v \in S$ et l'arrêt de destination $d \in S$. La distance est déterminée par la distance d'**Haversine** [10] entre les deux arrêts et pour en déterminer l'estimation du temps, nous utilisons une vitesse moyenne qui a été déterminée sur un échantillon de 60 voyages à des vitesses variées (voir graphe 1). Nous avons observé deux choses. Premièrement, une vitesse faible n'influence pas les petits voyages, tandis que nous obtenons de mauvais résultats pour de plus longues distances. Cela s'explique que sur de plus longues distances, nous prenons des moyens de transport plus rapides, augmentant la vitesse moyenne du voyage. Deuxièmement, nous avons observé un palier de vitesse après lequel le temps parcouru ne s'améliore plus. Ce palier est vite atteint par les plus courtes distances, tandis que celui-ci se stabilise vers 20m/s pour les distances plus élevées. Les valeurs dépassant ce palier tendent vers un résultat qui ressemble à **Dijkstra** puisque l'heuristique calculée devient trop faible par rapport au coût g . Nous avons donc pensé à un système pour déterminer l'heuristique dynamiquement selon la distance entre les deux arrêts. Nous déterminons la vitesse moyenne par palier de distance, au plus la distance est grande, au plus la vitesse l'est aussi (bien que nous bornons celle-ci par sécurité). Nous avons deux calculs d'heuristique différents, le premier est celui avec la vitesse palier fixe de tandis que le second est avec la vitesse dynamique selon les paliers de distance :

$$\begin{aligned}
h_statique(n) &= \frac{distance(n, destination)}{vitesse} \\
h_dynamique(n) &= a \cdot distance(n, destination)^b
\end{aligned}$$

Les paramètres a et b ont été déterminé après une étude d'un benchmark sur 60 voyages choisis aléatoirement. Cette deuxième fonction est moins robuste en terme de confiance mais assez intéressante, car elle résulte d'une approximation des résultats et nous donne une idée de comment l'heuristique influence l'optimalité du chemin en fonction de la distance. Nous pouvons nous demander si cette heuristique est admissible ($h(n) \leq c(n)$ où $c(n)$ est le coût réel) et même consistante ($h(n) \leq c(p, n) + h(p)$ où $c(p, n)$ est le coût pour atteindre n depuis p et où p est le successeur de n) [8].

Nous avons défini plusieurs méthodes pour évaluer l'admissibilité et la consistance de l'heuristique (voir section 7 et le pseudo-code 4). Tant pour la fonction $h_dynamique(n)$ que pour la fonction $h_statique(n)$ les résultats obtenus restent satisfaisants. En utilisant une vitesse moyenne optimiste, calculée dans notre échantillon (environ 20/s), nous obtenons de meilleurs résultats en termes de constance. En effet, en retraçant le chemin optimal et en évaluant l'admissibilité de chaque heuristique, nous constatons qu'elles sont en majorités consistantes. Cela n'est pas le cas pour la fonction $h_dynamique(n)$ qui voit sa consistance varier mais pas son admissibilité. Ces observations doivent être nuancées, car un échantillon de voyages ne couvre pas tous les cas possibles.

En conclusion, bien que notre heuristique ne soit pas optimale (par manque de temps et d'expérience), elle permet tout de même d'améliorer la performance en terme de vitesse d'exécution, corrélée au nombre de nœuds étudiés. Jusqu'à présent, nous n'avons pas trouvé de résultats qui possédaient des heuristiques non admissibles, et la plupart étaient même consistantes. Tout ce qui vient d'être énoncé peut être testé (section 7).

Algorithm 4: validateHeuristic

Data: start, end, departureTime, stops
Result: List of PathEdge

```

1 initialize A* with stops;
2 set Dijkstra mode to true;
3 optimalPath  $\leftarrow$  searchPath(start, end, departureTime);
4 set Dijkstra mode to false;
5 if optimalPath is empty then
6   | print("No optimal path found");
7   | return optimalPath;
8 end
9 isAdmissibleHeuristic  $\leftarrow$  true;
10 isConsistentHeuristic  $\leftarrow$  true;
11 for node  $\in$  optimalPath do
12   | from  $\leftarrow$  node.fromStop();
13   | to  $\leftarrow$  node.toStop();
14   | travelTime  $\leftarrow$  node.arrival() - node.departureTime();
15   | hN  $\leftarrow$  heuristic(to, end);
16   | hP  $\leftarrow$  heuristic(from, end);
17   | realCost  $\leftarrow$  calculateRealCost(to, end, node.departureTime(), stops);
18   | if  $hN \leq \text{travelTime} + hP$  then
19     | continue;
20   | end
21   | else if  $hN \leq \text{realCost}$  then
22     | isConsistentHeuristic  $\leftarrow$  false;
23   | end
24   | else
25     | isAdmissibleHeuristic  $\leftarrow$  false;
26   | end
27 end
28 print(isAdmissibleHeuristic);
29 print(isConsistentHeuristic);
30 return optimalPath;

```

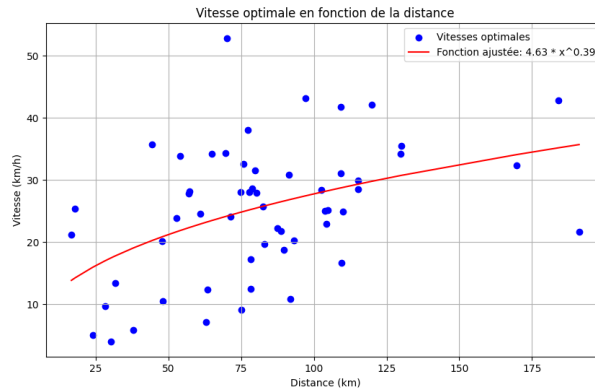


FIGURE 1 – Vitesse optimale estimée selon la distance (échantillon aléatoire sur 60 arrêts)

4.3 Analyse de complexité

Après cette longue discussion sur l'heuristique, nous allons expliquer en détail la complexité temporelle et spatiale de notre algorithme. Nous allons aussi présenter la complexité totale de notre algorithme, en comprenant aussi la reconstruction du chemin.

4.3.1 Complexité temporelle de l'algorithme

- **Expansion du graphe** Le calcul de l'heuristique se fait en temps constant, les opérations sur les *HashMap* aussi. Les opérations comme `contains` et `keyOf` sur `IndexMinPQ` sont aussi en temps constant, mais `insert` et `decreaseKey` se font en temps logarithmique $\mathcal{O}(\log V)$ où V est le nombre d'arrêts dans la queue et avec $V \subseteq S$ le nombre d'arrêts total. Complexité de l'opération : $\mathcal{O}(\log V) = \mathcal{O}(\log S)$ (si on ne considère uniquement cette opération, nous pouvons même dire de la complexité que c'est un $\Theta(\log V)$);
- **Essai des voisins** Sa complexité dépend du degré de voisins D où $|D| \ll |S|$ que nous parcourons. Tous les calculs simples sont effectués en temps constant. La complexité est aussi déterminée par l'appel à `tryExpand` qui étend le graphe, sa complexité a été déterminée au point précédent et est $\mathcal{O}(\log S)$, bien que parfois, le coût du trajet étant moins bon, cette méthode est en temps constant (nous pouvons dire que dans le meilleur des cas, la complexité est en $\Omega(D + \log V)$. Dans le pire des cas, la complexité est de $\mathcal{O}(D * \log S)$. Complexité de l'opération : $\mathcal{O}(D * \log S)$;
- **Recherche du chemin** La complexité est fortement variable, dans le meilleur des cas, on trouve directement l'arrêt final, nous donnant une complexité de $\Omega(D * \log V)$, car l'arrêt final se trouve dans les arrêts voisins de l'arrêt de départ et sa priorité sera la plus faible. Dans le pire des cas, cela dépendra directement de la distance entre les deux arrêts et de leur accessibilité. Au plus ils sont éloignés et peu accessibles, au plus nous devrons parcourir de noeuds. La complexité est donc $\mathcal{O}(S * D * \log S)$ car nous devons alors passer en revue tous les arrêts jusqu'au dernier, cas qui n'arrivera jamais car si nous parcourions tous les arrêts possibles, cela voudrait dire qu'on a trouvé un chemin jusqu'au dernier arrêt possible, ce qui est contradictoire. Complexité de l'opération : $\mathcal{O}(S * D * \log S)$;
- **Reconstruction du parcours** La reconstruction du parcours est directement proportionnelle au nombre d'arrêts parcourus. Pour un nombre d'arrêt parcourus P , nous faisons $P - 1$ boucles pour créer la liste et ensuite $P - 1$ opérations pour l'inverser, donc nous avons une complexité de $\mathcal{O}(P - 1)$. Complexité de l'opération : $\mathcal{O}(P)$;

Nous avons rassemblé toutes les étapes de notre algorithme et avons étudié leur complexité. Voici la complexité totale

$$\mathcal{O}(S \cdot D \cdot \log S + P)$$

Ce qui est une complexité classique pour un algorithme A*.

4.3.2 Complexité spatiale de l'algorithme

La complexité spatiale est plus simple à calculer. En effet, étant donné que nous créons uniquement les `PathEdge`, nous pouvons dire que la complexité spatiale de l'algorithme est :

$$\mathcal{O}(S + E)$$

où E est le nombre de `PathEdge` qui ont été créés.

4.3.3 Complexité temporelle et spatiale du programme

Nous avons déjà calculé la complexité temporelle et spatiale des `stop_times` qui était de $\mathcal{O}(L + T \cdot \text{Slog}S)$ et $\mathcal{O}(T \cdot S)$ où L est le nombre de lignes total du fichier CSV, T est le nombre de `Trip` et S le nombre de `Stop`. Calculons la complexité des autres *parsers* (l'étape du *parsing* est identique pour tous) :

- **Route** : Temporelle $\rightarrow \mathcal{O}(L)$ et Spatiale $\rightarrow \mathcal{O}(R)$ où R est le nombre de routes.
- **Trip** : Temporelle $\rightarrow \mathcal{O}(L)$ et Spatiale $\rightarrow \mathcal{O}(T)$ où T est le nombre de trajets.
- **Stop** La complexité des arrêts est un peu plus complexe, nous créons des liaisons à pied. Le *parsing* est identique aux autres, de complexité temporelle $\mathcal{O}(L)$ et spatiale $\mathcal{O}(S)$ où S est le nombre d'arrêts. Mais ensuite nous construisons un **R-Tree** [2] dont la complexité est $\mathcal{O}(\text{Slog}S)$. Enfin, nous construisons les liaisons à pied, pour cela, nous itérons sur chaque arrêt et nous récupérons voisins trouvés dans la zone du **R-Tree** en $\mathcal{O}(\log S + N)$ où N est le nombre d'arrêts voisins trouvés, puis nous itérons dans N pour créer les liaisons en $\mathcal{O}(N)$. Donc la complexité temporelle totale est $\mathcal{O}(S) + \mathcal{O}(\text{Slog}S) + \mathcal{O}(\log S + N) = \mathcal{O}(\text{Slog}S + N)$ et la complexité spatiale est $\mathcal{O}(S + N)$ car nous créons $|S|$ arrêts et $|N|$ liaisons de voisins.

En conclusion, la complexité du *parsing* de tous les fichiers GTFS a une complexité temporelle et spatiale totale :

$$\begin{aligned} &\mathcal{O}(L + T \cdot \text{Slog}S + N) \text{ (temporelle)} \\ &\mathcal{O}(T \cdot S + R + N) \text{ (spatiale)} \end{aligned}$$

Il nous reste qu'à associer cette complexité à celle précédemment calculée pour l'algorithme et d'y ajouter le parcours du chemin pour le *print* et nous nous retrouvons avec :

$$\begin{aligned} &\mathcal{O}(L + (T + D) \cdot \text{Slog}S + P) \text{ (temporelle)} \\ &\mathcal{O}(T \cdot S + E + R + N) \text{ (spatiale)} \end{aligned}$$

5 Gestion des variantes du problème

Nous avons aussi du implémenter des variants. Nous avons implémenté un système de profils, choisi par l'utilisateur au lancement du programme. Ces profils vont modifier le poids des trajets, du temps d'attente et du nombre de transferts effectués (seuls les transferts de routes sont comptabilisés). Les profils actuels sont :

- **NONE** Mode par défaut, le poids de chaque mode de transport, du temps d'attente et des transferts est à 1.0, cela veut dire que nous utilisons les valeurs initiales ;
- **COMFORTABLE** Mode qui privilégie le confort, cela veut dire que le poids du train et du métro sont avantagés par rapport aux autres transports, le temps d'attente n'est pas pénalisé, les transferts et la marche sont forts pénalisés ;

- **WALKING_FRIENDLY** Mode qui dépénalise la marche à pieds, pouvant donner des trajets plus longs que celui optimal avec le mode **NONE** où l'utilisateur marche plus. Le temps d'attente est un pénalisé, tout comme les autres transports et le nombre de transferts est aussi dépénalisé ;
- **FAST** Mode qui met en avant les transports, désavantage un peu la marche et ne pénalise pas les transferts.

Nous n'en avons pas rajouté d'autres, mais il est possible d'ajouter des profils personnalisés lors du lancement de l'algorithme.

6 Modélisation de la marche

La modélisation de la marche est assez simple et a partiellement déjà été expliquée lors du calcul de la complexité de **Stop** (voir section 4.3.3). Nous utilisons un R^* -Tree, une structure de donnée spécialisée dans l'indexation spatiale [4] et très efficace pour un nombre de données élevée. Nous utilisons la version R^* pour plus de précision grâce à un overlap plus précis. Nous insérons toutes nos données d'un coup (*bulk insertion*) pour gagner en performance. Nous avons trouvé une implémentation pour ce R^* -Tree sur github [2]. La distance maximale possible entre les arrêts est variable (selon l'opérateur), mais par défaut, nous les avons tous mis à 500 mètres. Nous avons observé qu'au delà de 5000 mètres, la création des voisins prend plusieurs secondes nuisant le confort utilisateur. Il est tout à fait possible d'associer des distances différentes selon l'opérateur. Cela permet en outre de diminuer la charge de voisin dans les zones très denses comme les villes en abaissant la limite de le STIB par exemple (réseau bruxellois).

Chaque **Neighbour** contient une **Trip** identique, c'est à dire que chaque liaison à pieds est considérée comme identique (exépté pour le temps de déplacement) et est donc facilement comparable.

7 Tests

Pour plus d'informations sur les tests, le fichier **README** du répertoire vous sera d'une grande aide. Concentrons nous plus sur l'analyse des résultats obtenus en comparant tout d'abord les résultats de **Dijkstra** avec **A***. Nous observons que les résultats obtenus sont très satisfaisant pour des courtes à moyennes (voir figure 2). L'avantage de notre algorithme se stabilise lorsque la distance devient grande.

Pour plus de tests, les fichiers csv générés sont disponibles dans le dossier **src/test/resources** et les instructions de tests dans le fichier **README**

Trajet #	Distance (km)	Temps de voyage	Nœuds développés
1	273,2	06 :55 :00	58521
2	48,1	00 :22 :54	4060
3	52,8	00 :57 :59	5394
4	1,8	00 :04 :00	53
5	428,4	15 :08 :00	64334
6	4,3	00 :12 :34	738
7	5,9	00 :15 :00	1311
8	171,1	00 :44 :49	1941
9	55,4	01 :11 :00	22188
10	170,9	03 :55 :00	52506
11	5,0	00 :19 :22	1237

TABLE 7 – Algorithme avec **Dijkstra** (départ 08 :00 :00)

Trajet #	Distance (km)	Temps de voyage	Nœuds développés
1	273,2	06 :55 :00	52547
2	48,1	00 :22 :54	1271
3	52,8	00 :57 :59	2142
4	1,8	00 :04 :00	20
5	428,4	15 :08 :00	64275
6	4,3	00 :12 :34	355
7	5,9	00 :15 :00	779
8	171,1	00 :44 :49	143
9	55,4	01 :11 :00	14167
10	170,9	03 :55 :00	34487
11	5,0	00 :19 :22	603

TABLE 8 – Algorithme avec **A*** (départ 08 :00 :00)

- 1 : Alveringem Nieuwe Herberg -> Aubange
- 2 : Brugge -> Brussel
- 3 : Antwerpen -> Gent
- 4 : Bruxelles-Midi -> Bruxelles-Central
- 5 : Paris Nord -> Amsterdam Cs
- 6 : schuman -> simonis
- 7 : etterbeek -> schaerbeek
- 8 : genk -> ostende
- 9 : liege -> namur
- 10 : delta -> aubange
- 11 : ulb -> rogier

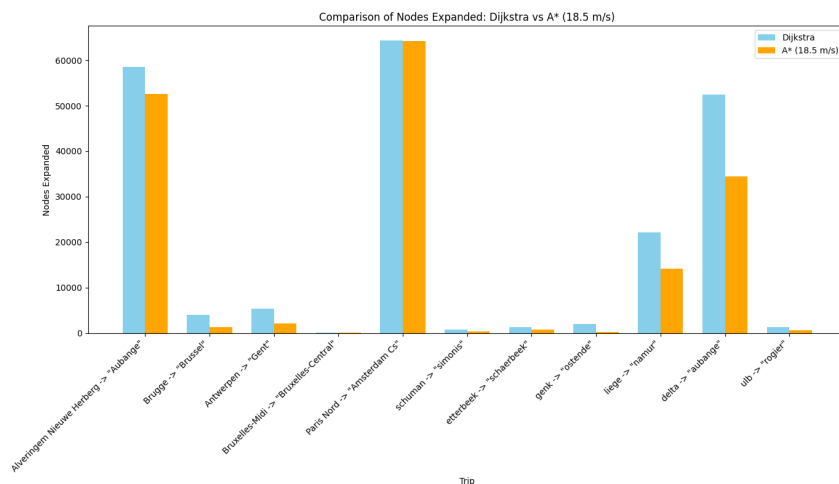


FIGURE 2 – Comparaison du nombre de noeuds parcourus

8 Conclusion

Nous arrivons sur la fin de cette aventure. Nous avons passé beaucoup de temps à réfléchir sur la question de l'heuristique et sa correcte forme, nous en avons conclu qu'une valeur statique restait préférable, mais que la version dynamique amenait à plus de questions et de possible optimisations. Nous avons dans l'ensemble une complexité classique, répondant correctement aux demandes.

Références

- [1] BAELDUNG. *Dijkstra vs. A* – Pathfinding*. [Online; accessed 12-May-2025]. 2021. URL : <https://www.baeldung.com/cs/dijkstra-vs-a-pathfinding>.
- [2] *rtree*. <https://github.com/davidmoten/rtree>. [Online; accessed 8-May-2025]. 2024.
- [3] Robert SEDGEWICK et Kevin WAYNE. *Algorithms, 4th Edition Textbook Code and Libraries*. <https://algs4.cs.princeton.edu/code/>. [Online; accessed 10-May-2025].
- [4] UNIVERSITÉ GUSTAVE EIFFEL. *Spatial Indexes - Structures*. [Online; accessed 18-May-2025]. 2014. URL : https://monge.univ-mlv.fr/~dr/XPOSE2013/spatial_indexes/page_structure.html#rstartree.
- [5] *univocity-parsers*. <https://github.com/univocity/univocity-parsers>. [Online; accessed 5-May-2025]. 2021.
- [6] WIKIPEDIA. *A* search algorithm* — *Wikipedia*. [Online; accessed 12-May-2025]. 2002. URL : https://en.wikipedia.org/wiki/A*_search_algorithm.
- [7] WIKIPEDIA. *Admissible heuristic* — *Wikipedia*. [Online; accessed 14-May-2025]. 2007. URL : https://en.wikipedia.org/wiki/Admissible_heuristic#cite_note-Misconceptions-3.
- [8] WIKIPEDIA. *Consistent heuristic* — *Wikipedia*. [Online; accessed 15-May-2025]. 2007. URL : https://en.wikipedia.org/wiki/Consistent_heuristic.

-
- [9] WIKIPEDIA. *Dijkstra's algorithm* — *Wikipedia*. [Online; accessed 12-May-2025]. 2002. URL : https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
 - [10] WIKIPEDIA. *Formule de haversine* — *Wikipédia*. [Online; accessed 2-May-2025]. 2017. URL : https://fr.wikipedia.org/wiki/Formule_de_haversine.